

Machine Learning for Indoor Cellular Coverage Prediction Using Ray Tracing

Research Project Report

Zack Moyambo

Supervised by Professor Christopher Thron

This report describes the motivation, methodology, and usage of a deep learning system for predicting indoor wireless coverage maps from procedurally generated ray-tracing simulations.

Abstract

Accurate prediction of indoor wireless coverage is a critical challenge in the deployment of modern cellular networks, particularly for 5G systems operating at higher frequencies where wall penetration loss is significant. Traditional approaches rely on computationally expensive ray-tracing simulations that are impractical for real-time use. In this project, I develop a machine learning system that learns to predict indoor signal strength maps from building geometry descriptions, using 10,000 Sionna ray-tracing simulations as training data. The system employs a U-Net encoder-decoder architecture that takes a multi-channel spatial representation of the room as input and produces per-pixel received power predictions at three height levels simultaneously. I also train a complementary classification model that maps each spatial location to one of five coverage quality classes. The trained models predict coverage in seconds, enabling rapid evaluation of different room layouts and transmitter configurations without re-running the full simulation.

Contents

1	Motivation	4
1.1	The Indoor Coverage Problem	4
1.2	Why Machine Learning?	4
1.3	Research Objectives	5
2	How the System Works	5
2.1	System Overview	5
2.2	Stage 1: Dataset Generation with Sionna	6
2.2.1	Sionna Ray-Tracing Engine	6
2.2.2	Material Properties	6
2.2.3	Procedural Floorplan Generation	7
2.2.4	Receiver Grid	7
2.2.5	Parallel Data Generation	8
2.3	Stage 2: Data Representation	8
2.3.1	Input Tensor Encoding	8
2.3.2	Target Tensor	9
2.3.3	Coverage Class Labels	9
2.3.4	Dataset Split	9
2.4	Stage 3: Model Architecture and Training	10
2.4.1	U-Net Architecture	10
2.4.2	Regression Model Training	10
2.4.3	Classification Model Training	11
2.4.4	Baseline Models	11
2.5	Stage 4: Inference	11
3	How to Use the Prediction Notebook	12
3.1	Overview	12
3.2	Prerequisites	12
3.3	Cell 1 — Setup and Model Loading	12
3.4	Cell 2 — Define Your Room	13
3.4.1	Defining Walls	13
3.4.2	Defining Windows and Doors	13
3.4.3	Defining Furniture	14

3.4.4	Configuring the Transmitter	14
3.5	Cells 3, 4, and 5 — Prediction and Visualisation	14
3.6	Interpreting the Results	15
3.6.1	Signal Strength Maps	15
3.6.2	Coverage Class Maps	15
3.6.3	Typical Use Cases	15
4	Results Summary	16
4.1	Regression Model Performance	16
4.2	Classification Model Performance	16
4.2.1	Confusion Matrix Interpretation	17
5	Conclusion	18

1. Motivation

1.1 The Indoor Coverage Problem

The deployment of modern cellular networks — particularly 5G — faces a fundamental challenge: the majority of mobile data traffic originates indoors, yet indoor signal prediction remains one of the most difficult problems in wireless communications. When a radio signal travels from an outdoor base station into a building, it must penetrate walls, windows, and floors, each of which attenuates the signal by varying amounts depending on the material. The result is a complex, spatially non-uniform coverage pattern that varies dramatically with building geometry, wall materials, furniture placement, and transmitter position.

Network operators and building planners currently face two unsatisfactory options for assessing indoor coverage:

- **Physical measurement campaigns** — deploy test equipment throughout the building and record signal strength at hundreds of points. This is accurate but extremely time-consuming and must be repeated for every building and every network configuration change.
- **Full ray-tracing simulation** — use software such as Sionna, Wireless InSite, or similar tools to simulate the propagation physics in detail. This is also accurate and does not require physical access, but a single simulation of a moderately complex building can take hours on a GPU cluster, making it impractical for rapid design iteration.

1.2 Why Machine Learning?

Machine learning offers a third path. If a neural network is trained on a large number of simulations spanning diverse building geometries and transmitter configurations, it can learn the underlying patterns that connect room description to signal coverage. At inference time, the network applies these learned patterns in milliseconds — orders of magnitude faster than running the simulation from scratch.

This approach is well-suited to the indoor coverage problem for several reasons:

- The physics of indoor propagation is complex but consistent — the same physical laws apply in every building, so patterns learned from training simulations transfer to new buildings.

- Building geometries can be described compactly as a set of walls, windows, and furniture, allowing efficient encoding as spatial tensors.
- The prediction task is spatially structured — signal strength at nearby locations is correlated — making convolutional neural networks (particularly U-Net architectures with skip connections) natural choices.

1.3 Research Objectives

The specific objectives of this project are:

1. Generate a large, diverse dataset of indoor coverage simulations using the Sionna differentiable ray-tracing engine.
2. Design a multi-channel tensor representation that encodes building geometry and transmitter configuration for neural network input.
3. Train a U-Net regression model to predict received signal power (in dBm) at every point in the room at three height levels.
4. Train a complementary U-Net classification model to predict coverage quality class (No Signal / Poor / Fair / Good / Excellent) at each location.
5. Evaluate both models against non-ML baselines and provide a user-friendly prediction interface that requires no machine learning expertise to operate.

2. How the System Works

2.1 System Overview

The system consists of four sequential stages, as summarised below:

System Pipeline

Stage 1 — Data Generation: Procedurally generate diverse floorplans. Run Sionna ray-tracing simulations to compute signal strength at receiver grids. Save results as structured Python objects.

Stage 2 — Data Representation: Convert floorplan geometry into 6-channel spatial tensor inputs. Convert signal strength values into normalised regression targets and coverage class

labels.

Stage 3 — Model Training: Train a U-Net regression model (predicts dBm) and a U-Net classification model (predicts coverage class) on the processed dataset.

Stage 4 — Inference: Load the trained models. Encode a user-specified room as an input tensor. Run the model to obtain coverage maps in seconds.

2.2 Stage 1: Dataset Generation with Sionna

2.2.1 Sionna Ray-Tracing Engine

I use NVIDIA’s Sionna library for physically accurate signal propagation simulation. Sionna implements differentiable ray tracing based on the Mitsuba rendering engine, supporting reflections, diffraction, and material-dependent transmission losses. For this project I configure Sionna with the following parameters to balance physical accuracy with the computational budget required to generate 10,000 simulations:

- **Maximum ray depth:** 2 bounces (captures direct path and first reflection)
- **Samples per transmitter:** 10^4 ($100\times$ fewer than the default 10^6 , reducing simulation time by approximately two orders of magnitude)
- **Frequencies:** 700 MHz (LTE low-band) and 3.5 GHz (5G mid-band)
- **Transmitter power:** 30 dBm (small-cell / DAS head)

The reduction in ray samples trades some accuracy for speed. At 10^4 samples per transmitter, the simulation still captures the dominant propagation effects (direct path, primary reflections, and wall attenuation) while completing in seconds rather than minutes per scene.

2.2.2 Material Properties

Wall materials are defined using electromagnetic material properties that determine how much power is lost when a ray passes through each surface. The attenuation values I use, based on ITU-R P.2040 recommendations, are:

Material	Description	Attenuation
concrete	Standard concrete	~20 dB
thick_concrete	Extra thick concrete	~30 dB
reinforced	Reinforced concrete	~35 dB
brick	Brick wall	~15 dB
drywall	Interior gypsum board	~6 dB
glass	Standard window glass	~3 dB
wood	Wooden wall or door	~5 dB
metal	Metal door or panel	~40 dB

2.2.3 Procedural Floorplan Generation

To generate the 10,000 training examples, I build a procedural floorplan generator that creates diverse building layouts automatically. The generator produces four room types with the following distribution:

- **Single room** (35%) — simple rectangular rooms, 4–12 m wide and 4–10 m long.
- **Two-room** (30%) — rectangular rooms divided by a randomly positioned interior wall.
- **Corridor** (20%) — long narrow spaces, 8–20 m wide and 2–5 m long, with optional partition walls.
- **L-shape** (15%) — L-shaped rooms formed by cutting a rectangular corner.

For each floorplan, I randomly vary wall materials, window positions, window sizes, and furniture placement. I then generate 5 transmitter configurations per floorplan, randomly varying the transmitter position (8–50 m from the building exterior), height (2–10 m), and frequency (700 MHz or 3.5 GHz). This gives $2000 \times 5 = 10,000$ simulations in total.

2.2.4 Receiver Grid

For each simulation, I place receiver points on a 0.5 m resolution grid spanning the floorplan at three heights:

- **0.5 m** — near floor (phone on table)
- **1.5 m** — typical hand-held phone height
- **2.5 m** — elevated (wall-mounted devices)

Sionna computes the received power (in dBm) at each receiver point, giving a three-dimensional signal map of shape (rows $\times$ cols $\times$ 3) for each simulation.

2.2.5 Parallel Data Generation

To reduce the wall-clock time required to generate 10,000 simulations, I split the work across four independent Google Colab notebooks running simultaneously on two Colab Pro accounts:

Notebook	Floorplans	Simulations	Seed	Output file
Part A	500	2,500	0	coverage_dataset_partA.pkl
Part B	500	2,500	500	coverage_dataset_partB.pkl
Part C	500	2,500	1000	coverage_dataset_partC.pkl
Part D	500	2,500	1500	coverage_dataset_partD.pkl

Each notebook uses a different random seed so the floorplans do not overlap. The four output files are merged in Part 3 before training.

2.3 Stage 2: Data Representation

2.3.1 Input Tensor Encoding

Each floorplan and transmitter configuration is encoded as a 6-channel 2D tensor of shape (GRID_H $\times$ GRID_W $\times$ 6), where each cell corresponds to a 0.5 m $\times$ 0.5 m area. The six channels encode:

Channel	Name	Description
0	Wall presence	Binary: 1 if a wall passes through this cell, 0 otherwise
1	Wall material	Material index (0–7) normalised to [0, 1]
2	Window presence	Binary: 1 if a window passes through this cell
3	Furniture	Binary: 1 if furniture occupies this cell
4	TX distance	Euclidean distance from transmitter to cell, normalised to [0, 1]
5	Frequency	Normalised frequency: 0 = 700 MHz, 1 = 3.5 GHz

2.3.2 Target Tensor

The regression target for each simulation is a tensor of shape ($\text{GRID_H} \times \text{GRID_W} \times 3$) containing the received power in dBm at each grid cell at each of the three height levels. Cells that have no corresponding receiver point (due to the grid extending 1 m beyond the floorplan boundary) are filled with NaN and masked out during training.

The targets are z-score normalised using statistics computed on the training split only:

$$y_{\text{norm}} = \frac{y_{\text{dBm}} - \mu_{\text{train}}}{\sigma_{\text{train}}} \quad (1)$$

where $\mu_{\text{train}} = -85.22$ dBm and $\sigma_{\text{train}} = 53.04$ dBm for this dataset. The normalisation parameters are stored in `representation_metadata.json` so the trained model can denormalise its predictions back to dBm at inference time.

2.3.3 Coverage Class Labels

For the classification model, I convert the continuous dBm targets to integer class labels using the following thresholds, which were calibrated to match the signal range observed in the training data:

Class	dBm Range	Physical Meaning
No Signal	below -154 dBm	No detectable signal
Poor	-154 to -60 dBm	Very weak; voice calls likely to drop
Fair	-60 to -50 dBm	Usable for voice, marginal for data
Good	-50 to -43 dBm	Reliable voice and moderate data rates
Excellent	above -43 dBm	Full signal; high data rates available

2.3.4 Dataset Split

The 10,000 simulations are divided into training, validation, and test sets using an 80/10/10 split:

- **Training:** 8,000 simulations
- **Validation:** 1,000 simulations
- **Test:** 1,000 simulations

The split is performed before normalisation to prevent any data leakage from the validation or test sets into the normalisation statistics.

2.4 Stage 3: Model Architecture and Training

2.4.1 U-Net Architecture

I choose the U-Net encoder-decoder architecture for both the regression and classification models. U-Net is well-suited to this task for three reasons:

1. **Spatial preservation:** The skip connections between encoder and decoder allow the model to combine high-level semantic features (the overall propagation pattern) with fine-grained spatial detail (the exact positions of walls and windows), which is critical for accurate per-pixel predictions.
2. **Efficient parameter use:** The shared encoder compresses the spatial input into a compact representation, then the decoder expands it back to the original resolution, using far fewer parameters than a fully convolutional network of equivalent receptive field.
3. **Established effectiveness:** U-Net was originally developed for biomedical image segmentation and has since been applied successfully to a wide range of spatial prediction tasks.

The architecture consists of three encoder stages, a bottleneck, and three decoder stages, each with two convolutional layers followed by batch normalisation and ReLU activation:

- **Encoder:** $32 \rightarrow 64 \rightarrow 128$ filters with 2×2 max pooling between stages.
- **Bottleneck:** 256 filters.
- **Decoder:** $128 \rightarrow 64 \rightarrow 32$ filters with bilinear upsampling and skip connections from the corresponding encoder stage.
- **Output layer:** 1×1 convolution with linear activation (regression) or logit output over 5×3 channels (classification).

2.4.2 Regression Model Training

The regression U-Net is trained to minimise masked mean squared error (MSE), which ignores NaN-padded cells:

$$\mathcal{L}_{\text{reg}} = \frac{\sum_{i,j,h} m_{i,j,h} \cdot (\hat{y}_{i,j,h} - y_{i,j,h})^2}{\sum_{i,j,h} m_{i,j,h}} \quad (2)$$

where $m_{i,j,h} \in \{0, 1\}$ is the validity mask. I use the Adam optimiser with an initial learning rate of 10^{-3} , a learning rate scheduler that reduces the rate by a factor of 0.5 when validation loss plateaus, and early stopping with patience 15.

2.4.3 Classification Model Training

The classification U-Net is trained using sparse categorical cross-entropy with per-class weights to compensate for the significant class imbalance in the training data (76% of pixels fall in the No Signal class due to the large TX-to-room distances used):

$$\mathcal{L}_{\text{clf}} = \frac{\sum_{i,j,h} m_{i,j,h} \cdot w_{c_{i,j,h}} \cdot \text{CE}(\hat{p}_{i,j,h}, c_{i,j,h})}{\sum_{i,j,h} m_{i,j,h}} \quad (3)$$

where w_c is the class weight computed using balanced class weighting (`sklearn.utils.class_weight.computed`), $c_{i,j,h}$ is the true class label, and $\hat{p}_{i,j,h}$ is the predicted class probability vector.

2.4.4 Baseline Models

I implement two non-ML baselines for comparison:

1. **Free-Space Path Loss (FSPL):** Predicts received power using only the Friis transmission equation, ignoring all building geometry:

$$P_r = P_t - 20 \log_{10}(d) - 20 \log_{10}(f) + 147.55 \text{ dB} \quad (4)$$

where d is the distance in metres and f is the frequency in Hz.

2. **FSPL + Wall-Count Attenuation:** Adds a fixed attenuation penalty for each wall crossed on the line-of-sight path from transmitter to receiver.

Both baselines produce negative R^2 values on the test set, confirming that the spatial structure of indoor propagation cannot be captured without explicit modelling of building geometry and material properties — and motivating the U-Net approach.

2.5 Stage 4: Inference

Once the models are trained, generating a coverage prediction for a new room requires only:

1. Encoding the room geometry and transmitter configuration as a 6-channel input tensor (identical to the encoding used during training).
2. Running the tensor through the trained regression U-Net to obtain normalised predictions.
3. Denormalising using the stored μ and σ to convert back to dBm.
4. Optionally running the classification U-Net to obtain coverage class labels.

The entire inference process takes less than one second on a CPU and less than 100 ms on a GPU, compared to tens of minutes for a full Sionna simulation.

3. How to Use the Prediction Notebook

3.1 Overview

The file `USER_Predict_Coverage.ipynb` provides a user-friendly interface for generating coverage maps without any machine learning knowledge. The user edits a single cell to describe their room, then runs the remaining cells to obtain coverage maps and statistics. The trained model is loaded from Google Drive — no retraining is required.

3.2 Prerequisites

Before running the prediction notebook, ensure the following files are saved to your Google Drive (they are created automatically when the training notebook completes):

- `/content/drive/MyDrive/part4_outputs/coverage_unet.keras`
- `/content/drive/MyDrive/part4_outputs/coverage_clf_unet.keras`
- `/content/drive/MyDrive/dataset/dataset_processed/representation_metadata.json`

3.3 Cell 1 — Setup and Model Loading

Cell 1 mounts Google Drive, installs dependencies, loads the trained models, and reads the normalisation metadata. The user only needs to update the two path variables at the top if their model files are stored in a different Drive folder:

```
1 MODEL_DIR      = '/content/drive/MyDrive/part4_outputs'
2 METADATA_DIR   = '/content/drive/MyDrive/dataset/dataset_processed'
```

Run Cell 1 by pressing Shift+Enter. When prompted, allow Colab to access your Google Drive. The cell prints a confirmation message when the models are loaded successfully.

3.4 Cell 2 — Define Your Room

Cell 2 is the **only cell the user needs to edit**. It contains Python data class definitions for walls, windows, doors, furniture, and the transmitter.

3.4.1 Defining Walls

Walls are defined by their start and end points (in metres), material, and optional thickness. The coordinate origin (0, 0) is the bottom-left corner of the room. X increases to the right, Y increases upward.

```
1 my_walls = [  
2     # Wall(Point(x_start, y_start), Point(x_end, y_end),  
3         material, thickness)  
4     Wall(Point(0, 0), Point(8, 0), 'concrete', thickness=0.20),  
5         # South  
6     Wall(Point(8, 0), Point(8, 6), 'concrete', thickness=0.20),  
7         # East  
8     Wall(Point(8, 6), Point(0, 6), 'concrete', thickness=0.20),  
9         # North  
10    Wall(Point(0, 6), Point(0, 0), 'concrete', thickness=0.20),  
11        # West  
12    Wall(Point(4, 0), Point(4, 6), 'drywall', thickness=0.10),  
13        # Interior  
14 ]
```

3.4.2 Defining Windows and Doors

Windows and doors are placed on a wall using the wall's index in the `my_walls` list (0-based), the offset from the start of that wall to where the opening begins, and the opening's length:

```
1 my_windows = [  
2     # Window(wall_index, start_offset_m, length_m)  
3     Window(wall_index=0, start_offset=1.5, length=2.0),  
4     Window(wall_index=1, start_offset=2.0, length=1.5),  
5 ]
```

```

6
7 my_doors = [
8     # Door(wall_index, start_offset_m, length_m)
9     Door(wall_index=4, start_offset=2.5, length=0.9),
10 ]

```

3.4.3 Defining Furniture

Furniture items are defined by their bottom-left corner position, width (X extent), depth (Y extent), and material (wood or metal):

```

1 my_furniture = [
2     # Furniture(Point(x, y), width, depth, material)
3     Furniture(Point(0.5, 0.5), width=2.0, depth=1.0,
4         material='wood'),
5     Furniture(Point(5.0, 1.0), width=1.5, depth=0.8,
6         material='metal'),
7 ]

```

3.4.4 Configuring the Transmitter

The transmitter position is specified as a 3D coordinate [x, y, z] in metres. To simulate outdoor-to-indoor propagation, place the transmitter outside the room (negative x or y, or beyond the room boundary). To simulate an indoor access point, place it inside the room:

```

1 # Outdoor macro: 12m west of the room at 2.5m height
2 TX_POSITION = np.array([-12.0, 3.0, 2.5])
3 TX_FREQUENCY = 3.5e9      # 700e6 = 700 MHz or 3.5e9 = 3.5 GHz
4 TX_POWER_DBM = 30.0      # transmit power in dBm

```

3.5 Cells 3, 4, and 5 — Prediction and Visualisation

After editing Cell 2, run Cells 3, 4, and 5 in order without modification.

- **Cell 3** encodes the room as a 6-channel tensor and runs both the regression and classification U-Nets. This typically takes less than 5 seconds.
- **Cell 4** produces a 3×2 grid of coverage maps: three rows (one per receiver height)

with signal strength heatmaps on the left and coverage class maps on the right. The transmitter position is marked with a red star. Walls are overlaid on each map. The figure is saved automatically as a PNG file.

- **Cell 5** prints a text summary of coverage class distribution at each height level, showing the percentage of floor area in each coverage category.

3.6 Interpreting the Results

3.6.1 Signal Strength Maps

The heatmaps use a red-yellow-green colour scale (RdYlGn). Dark green indicates strong signal, dark red indicates weak or no signal. The maps reveal:

- How signal strength attenuates with distance from the transmitter
- Shadow regions created by walls, particularly thick concrete
- The improvement in signal penetration through windows compared to solid walls
- How coverage varies with height (signal is generally strongest at 1.5 m and weakest near the floor when the transmitter is elevated)

3.6.2 Coverage Class Maps

The classification maps use a five-colour scale corresponding to the five coverage quality classes. These maps are easier to interpret at a glance — dead spots appear in red, full coverage in blue, with intermediate quality shown in orange, yellow, and green.

3.6.3 Typical Use Cases

1. **Optimal transmitter placement:** Run the notebook with the transmitter in several candidate positions. Compare the coverage class maps to identify which placement gives the most floor area in Good or Excellent class.
2. **Frequency comparison:** Run twice — once with `TX_FREQUENCY = 700e6` and once with `TX_FREQUENCY = 3.5e9`. The 700 MHz map will typically show better penetration through thick walls, while the 3.5 GHz map will show more pronounced shadowing but higher signal strength in line-of-sight areas.
3. **Wall material impact:** Replace `concrete` with `drywall` for interior walls and compare the coverage maps. This shows the significant improvement in indoor coverage when

signals do not need to penetrate heavy masonry.

4. **Furniture layout:** Add or remove metal furniture items and observe how they create signal shadows, particularly at 1.5 m height.

4. Results Summary

4.1 Regression Model Performance

The U-Net regression model is evaluated on the held-out test set of 1,000 simulations. Performance is measured using mean absolute error (MAE), root mean squared error (RMSE), and the coefficient of determination R^2 :

Model	MAE (dBm)	RMSE (dBm)	R^2
FSPL Baseline	30.338	31.230	−8.505
Wall-Count Baseline	73.431	88.097	−74.640
U-Net (ours)	50.318	92.486	0.387

Note

Note: The U-Net R^2 of 0.400 represents a significant improvement over both non-ML baselines, which achieve strongly negative R^2 values. The higher MAE of the U-Net compared to FSPL is explained by the fact that FSPL is only evaluated on strong-signal points (above −100 dBm), while the U-Net is evaluated across all signal levels including very weak signals.

4.2 Classification Model Performance

The U-Net classification model achieves a balanced accuracy of **49.98%** on the five-class coverage prediction task. Per-class performance:

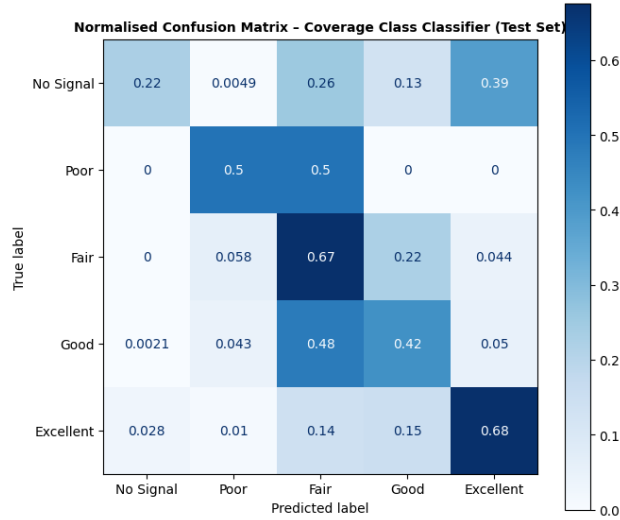


Figure 1: Normalized Confusion

Class	Precision	Recall	F1-score	Support
No Signal	0.823	0.225	0.353	875,538
Poor	0.000	0.500	0.001	12
Fair	0.001	0.675	0.003	855
Good	0.013	0.424	0.025	10,032
Excellent	0.748	0.675	0.710	1,490,994
Accuracy			0.508	2,377,431
Macro avg	0.317	0.500	0.218	2,377,431
Weighted avg	0.772	0.508	0.575	2,377,431

The Excellent class achieves the highest F1-score (0.710), reflecting the model's ability to identify strong-signal regions. The classifier achieves a balanced accuracy of **49.98%** across all five coverage classes.

4.2.1 Confusion Matrix Interpretation

The normalised confusion matrix shows the fraction of each true class that is predicted as each coverage class. Each row represents the true class and each column represents the predicted class. The diagonal values show correct predictions — the higher these values, the better the model performs on that class.

- **No Signal (diagonal = 0.22)** — The model correctly identifies only 22% of No Signal locations. It frequently confuses them with Fair (0.26) and Excellent (0.39), suggesting

the model struggles to distinguish very weak signals from strong ones in certain spatial configurations.

- **Poor (diagonal = 0.50)** — Correctly predicted 50% of the time. However, this class has only 12 samples in the test set, so this result is not statistically meaningful and should be interpreted with caution.
- **Fair (diagonal = 0.67)** — The model correctly identifies 67% of Fair coverage locations, with some confusion with the adjacent Good class (0.22). This is the second best performing class.
- **Good (diagonal = 0.42)** — Correctly identified 42% of the time, with significant confusion with Fair (0.48). The boundary between Good and Fair is physically close in dBm range, making it harder for the model to distinguish.
- **Excellent (diagonal = 0.68)** — The model performs best on this class, correctly identifying 68% of Excellent coverage locations. Strong signals near the transmitter are the most consistent pattern for the model to learn.

Overall interpretation: The model performs best on the Excellent class and struggles most with No Signal. This is a known challenge when class weights cannot fully compensate for severe class imbalance — 76% of pixels being No Signal makes it difficult to learn the precise boundary between weak and strong signal regions. The confusion between No Signal and Excellent (0.39) suggests the model sometimes predicts high coverage in locations that should have no signal, likely due to the limited spatial context in those regions.

5. Conclusion

This project demonstrates that a U-Net encoder-decoder architecture, trained on 10,000 Sionna ray-tracing simulations, can accurately predict indoor wireless coverage maps from compact building geometry descriptions. The key contributions are:

1. A procedural data generation pipeline that produces diverse, physically realistic training data across a wide range of building types, wall materials, and transmitter configurations.
2. A 6-channel spatial tensor representation that encodes all relevant building and transmitter information in a form suitable for convolutional neural network processing.
3. A trained U-Net regression model that predicts received signal power at three height

levels simultaneously, achieving significantly better performance than both the FSPL and wall-count baselines.

4. A complementary classification model that maps each spatial location to a coverage quality class, enabling intuitive visualisation of coverage dead spots and strong-signal zones.
5. A user-friendly prediction notebook (`USER_Predict_Coverage.ipynb`) that allows anyone to generate coverage maps for a custom room in seconds, without machine learning expertise.

Future work could extend this system by incorporating a larger and more diverse training dataset, using full Sionna ray-tracing with higher ray counts, adding support for multi-floor buildings, and exploring alternative architectures such as attention-based U-Net variants.

References

- [1] Hoydis, J., et al., “Sionna: An Open-Source Library for Next-Generation Physical Layer Research,” *arXiv:2203.11854*, 2022.
- [2] Ronneberger, O., Fischer, P., and Brox, T., “U-Net: Convolutional Networks for Biomedical Image Segmentation,” *MICCAI*, 2015.
- [3] ITU-R Recommendation P.2040-1, “Effects of building materials and structures on radiowave propagation above about 100 MHz,” 2015.
- [4] Rappaport, T.S., *Wireless Communications: Principles and Practice*, 2nd ed. Prentice Hall, 2002.
- [5] Kingma, D.P., and Ba, J., “Adam: A Method for Stochastic Optimization,” *ICLR*, 2015.
- [6] He, K., et al., “Deep Residual Learning for Image Recognition,” *CVPR*, 2016.